Web Technologies

Irshad Ullah
Department of Computer Science
COMSATS University Islamabad, Attock Campus

Table of contents

1. Rounded corners, gradients, shadows and opacity
2. 2D/3D transforms, transitions and keyframe animations
3. Web fonts, icon fonts and CSS custom properties (variables)
4. Media and feature queries
5. Vendor prefixes and cross-browser compatibility

What is CSS3?

CSS3 is the evolution of CSS that introduced many features for creating modern and interactive web pages.

CSS3 allows developers to create rounded elements, gradients, shadows, transformations, transitions, animations, custom fonts, responsive layouts and reusable variables.

Example: .card

```
{
  border-radius: 12px;
  box-shadow: 0 5px 15px #999;
}
```

Rounded Corners

The border-radius property creates rounded corners.

```
Syntax: border-radius: 20px;
```

```
Example: .card { width: 300px; padding: 20px; border: 2px solid
black; border-radius: 15px; }
```

Useful for cards, buttons, images, containers and forms.

Visual output

Rounded card

Different Border Radius Values

`border-radius: 20px;` → all corners

`border-radius: 10px 20px 30px 40px;` → top-left, top-right, bottom-right, bottom-left

Individual properties: border-top-left-radius, border-top-right-radius, border-bottom-right-radius, border-bottom-left-radius

Creating a Circle

A square can be turned into a circle using equal width and height with border-radius: 50%.

Example: .avatar

```
{
    width: 100px;
    height: 100px;
    border-radius:
    50%;
}
```

Visual output



For a perfect circle, width and height should normally be equal.

CSS Gradients

A gradient creates a smooth transition between two or more colors.

CSS provides linear, radial and conic gradients.

Linear example: background: linear-gradient(red, blue);

Gradients are commonly used for backgrounds, buttons, cards and visual effects.

Visual output



Linear Gradient

A linear gradient changes color along a line.

```
Example: background: linear-gradient(to right, red, blue);
```

Directions include to left, to right, to top and to bottom.

Angles can also be used: linear-gradient(45deg, red, blue).

Multiple colors are supported.

Visual output



red → yellow → green → blue

Radial Gradient

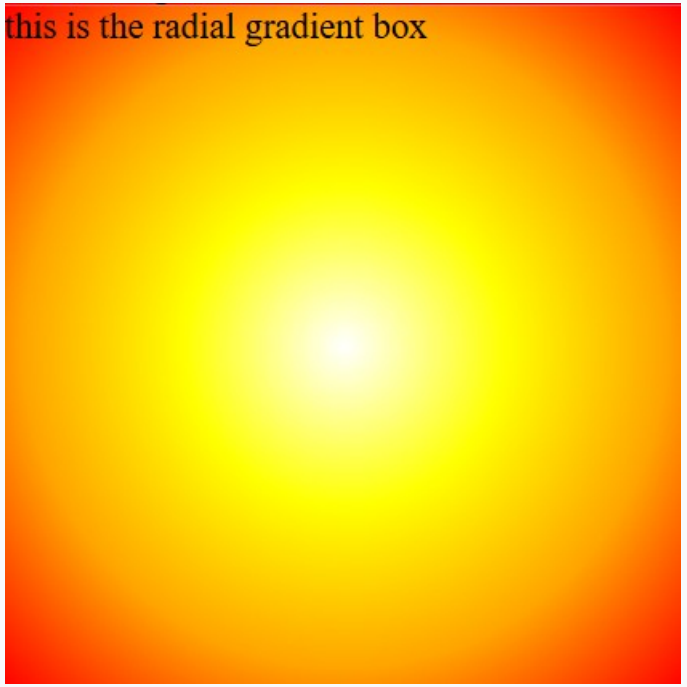
A radial gradient spreads outward from a central point.

```
Example: background: radial-gradient(circle, white, yellow, orange, red);
```

```
Linear gradient → color changes along a line.
```

Radial gradient → color spreads outward from a center.

Visual out



CSS Box Shadow

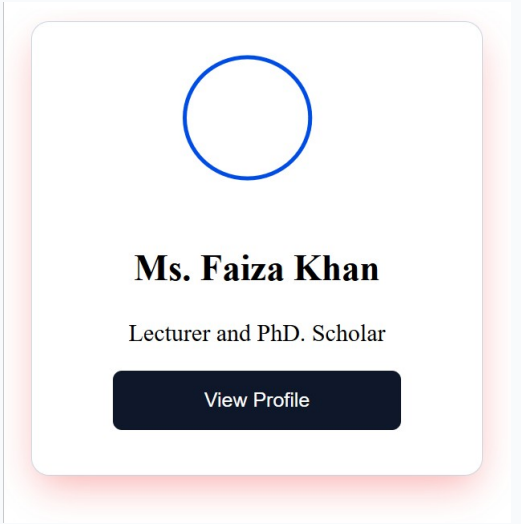
box-shadow adds a shadow around an element.

```
Syntax: box-shadow: x-offset y-offset blur spread color;
```

```
Example: box-shadow: 0 10px 20px gray;
```

Common uses: cards, buttons, navigation bars, modals and images.

Visual output



Text Shadow

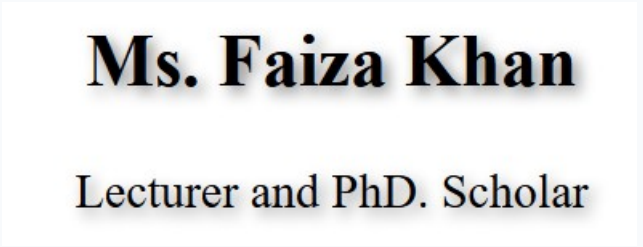
text-shadow adds a shadow to text.

```
Example: h1
          {
              text-shadow: 2px 2px 5px gray;
          }
```

box-shadow → shadow around an element.

text-shadow → shadow around text.

Visual output



Opacity

opacity controls how transparent an element is.

`0 → completely transparent`

`1 → completely visible`

`Example: img { opacity: 0.5; }`

An opacity of 0.5 makes the element 50% opaque.

Visual output



opacity 0.2 | opacity 0.5 | opacity 1(default)

CSS Transform

The transform property changes the appearance or position of an element without changing the normal document flow.

Common transformations: translate(), rotate(), scale(), skew().

```
Example: .box { transform: rotate(20deg); }
```

Visual output



Translate

translate() moves an element.

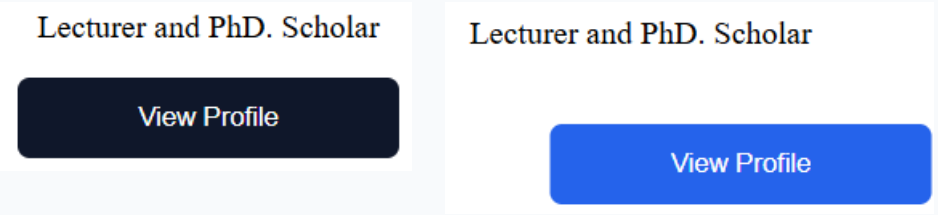
```
Example: transform: translate(50px, 20px);
```

translateX(50px) → horizontal movement.

translateY(20px) → vertical movement.

```
Example: .box:hover { transform: translateY(-5px); }
```

Visual output



Scale

scale() changes the size of an element.

`1 → original size`

1.2 → 120% of original size

0.5 → 50% of original size

`Example: img:hover { transform: scale(1.1); }`

Visual output



Skew

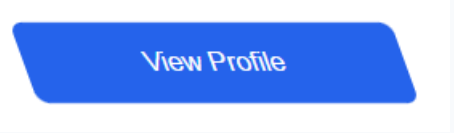
skew() slants an element.

```
Example: transform: skew(20deg);
```

skewX(20deg) → slant along X axis.

skewY(20deg) → slant along Y axis.

Visual output



2D vs 3D Transform

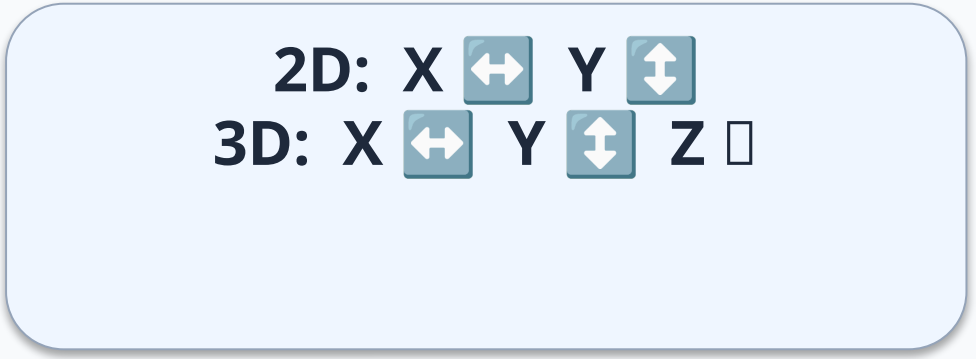
2D transformations mainly use the X and Y axes.

Examples: translate(), rotate(), scale(), skew().

3D transformations add the Z axis.

Examples: translateZ(), rotateX(), rotateY(), rotateZ().

Visual output



3D Transform Example

```
Example: .card:hover { transform: rotateY(180deg); }
```

This rotates the card around the Y-axis.

3D effects often work together with perspective.

```
Example: .container { perspective: 500px; }
```

CSS Transition

A transition creates a smooth change between two CSS states.

Without transition → the change happens immediately.

With transition → the change happens gradually.

Example:

```
button {
    background: black;
    transition: 0.3s;
            }
button: hover {
    background: blue;
            }
```

Transition Properties

transition-property → property being animated

transition-duration → length of the transition

transition-timing-function → speed pattern

transition-delay → waiting time before the transition

Shorthand: transition: background 0.3s ease;

Common timing functions: ease, linear, ease-in, ease-out, ease-in-out.

Transform + Transition

A common modern UI effect combines transform and transition.

```
Example: .button { transition: all 0.2s ease; }
.button:hover { transform: translateY(-2px); }
```

Transition controls how smoothly the transform occurs.

CSS Keyframe Animation

CSS animations allow an element to change through multiple stages.

They use @keyframes.

```
<h1>The @keyframes Rule</h1>

<div>i am moving</div>
```

```
@keyframes mymove {
  from {top: 0px;}
  to {top: 200px;}
}
div {
  width: 100px;
  height: 100px;
  background: red;
  position: relative;
  animation: mymove 2s infinite;
}
```

Animation Properties

An animation lets an element gradually change from one style to another.
To use CSS animation, you must specify some keyframes for the animation.

Important properties include animation-name, animation-duration, animation-delay, animation-iteration-count, animation-direction and animation-timing-function.

```
Example: .box {

animation-name: move;

animation-duration: 2s;

animation-iteration-count: infinite;

}
```

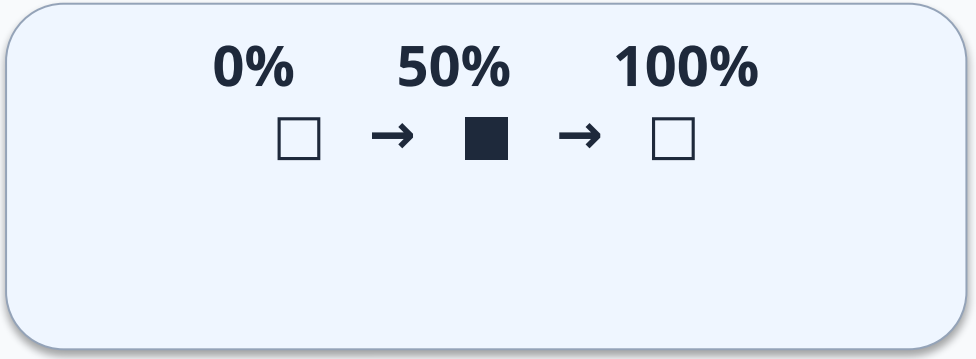
Multiple Keyframes

Animations can use percentages instead of only from and to.

`Example: 0% → start, 50% → move, 100% → return.`

This allows multi-step motion and more complex effects.

Visual output



Transition vs Animation

Transition usually requires a state change such as :hover.

Animation can run automatically using @keyframes.

Transform changes the visual state; transition makes a change smooth; animation defines multiple stages over time.

Visual output

Transition: hover
Animation: automatic

Web Fonts

Web fonts allow websites to use fonts that may not be installed on the user's computer.

@font-face defines a web font.

```
Example: @font-face {
font-family: MyFont;
src: url('myfont.woff2');
}
P{
font-family: MyFont;
}
```

Then use it with: body { font-family: MyFont; }

Font Formats

Common web font formats include WOFF, WOFF2, TTF and OTF.

WOFF2 is commonly preferred for modern websites because of efficient compression and broad modern browser support.

.woff2 .woff .ttf .otf

Icon Fonts

Icon fonts represent icons using font characters.

Examples include Font Awesome and Material Icons.

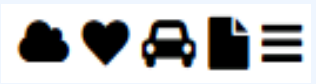
Icons can be resized and recolored using CSS.

Example concept:

```
<head>
<script src="https://kit.fontawesome.com/a076d05399.js" crossorigin="anonymous"></script>
</head>
<body>
<i class="fas fa-cloud"></i>
<i class="fas fa-heart"></i>
<i class="fas fa-car"></i>
<i class="fas fa-file"></i>
<i class="fas fa-bars"></i>

</body>
</html>
```

Visual output



CSS Custom Properties

- Also known as **CSS variables**
- Allow you to store specific values in one place and reuse them.
- When you define CSS variables
- To define a custom property, name it starting with two dashes (--).
- To use it, wrap the variable name inside the var() function

inside: root, they are generally available **throughout the entire webpage**.

Example:

```
:root { --primary-color:  #2563eb; }
```

Use with:

```
button {
 background: var(--primary-color);
}
```

Visual output



Why Use CSS Variables?

Without variables, the same color or value may be repeated throughout the stylesheet.

With variables, define the value once and reuse it.

If the value changes, update the variable rather than every individual rule.

This improves consistency and maintainability.

Change once → updates everywhere

Media Queries

Media queries apply different CSS depending on characteristics of the viewport or device.

They are commonly used for responsive web design.

```
Example: @media (max-width: 600px) { .profile-card { width: 100%; } }
```

Visual output

Desktop → Mobile
wide card → full-width card

Responsive Design

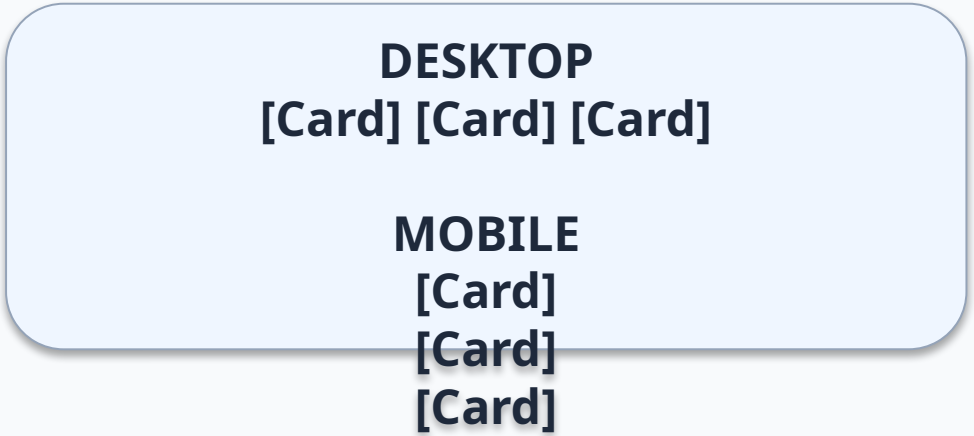
A responsive website adapts its layout to different screen sizes.

Desktop may display multiple cards in a row.

Mobile may stack cards vertically.

Media queries help create these layout changes.

Visual output



Media Query Syntax

CSS media queries allow you to apply styles based on the characteristics of a device or the environment displaying the web page.

The CSS @media rule is used to add media queries to your style sheet.

General syntax: @media media-type and (condition) { /* CSS rules */ }

```
Example: @media screen and (max-width: 768px) { body { font-size: 14px; } }
```

Common conditions: max-width, min-width, max-height, min-height and orientation.

Visual

≤ 768px → mobile styles

Feature Queries

Feature queries use @supports.

They check whether the browser supports a particular CSS feature.

```
Example: @supports (display: grid) { .container { display: grid; } }
```

They can be used to provide fallbacks for newer CSS features.

Visual output

Supports Grid? YES → Grid
NO → Fallback

Why Use @supports?

A fallback can be provided when a browser does not support a modern feature.

```
Example: .container { display: block; }
```

Then: @supports (display: grid) { .container { display: grid; } }

The browser uses Grid when supported and the fallback otherwise.

Visual output

Modern browser → Grid
Older browser → Block

Vendor Prefixes

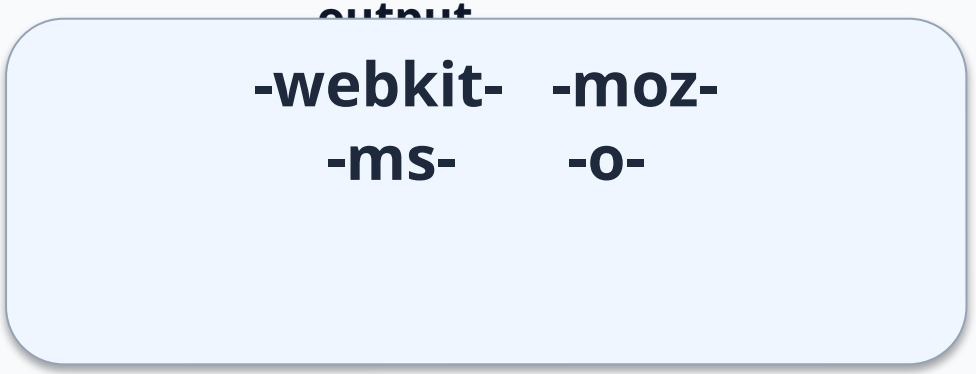
Vendor prefixes were used to add new experimental or browser-specific CSS features.

Examples: -webkit-, -moz-, -ms-, -o-

```
Example: -webkit-transform: rotate(20deg); transform: rotate(20deg);
```

Prefixes are less necessary for many modern CSS features.

Visual output



Are Vendor Prefixes Still Necessary?

Not usually for most modern CSS features.

Modern browsers support many CSS properties without prefixes.

Prefixes may still appear in legacy code or be relevant for certain experimental features.

Do not automatically add prefixes to every CSS property.

Visual output

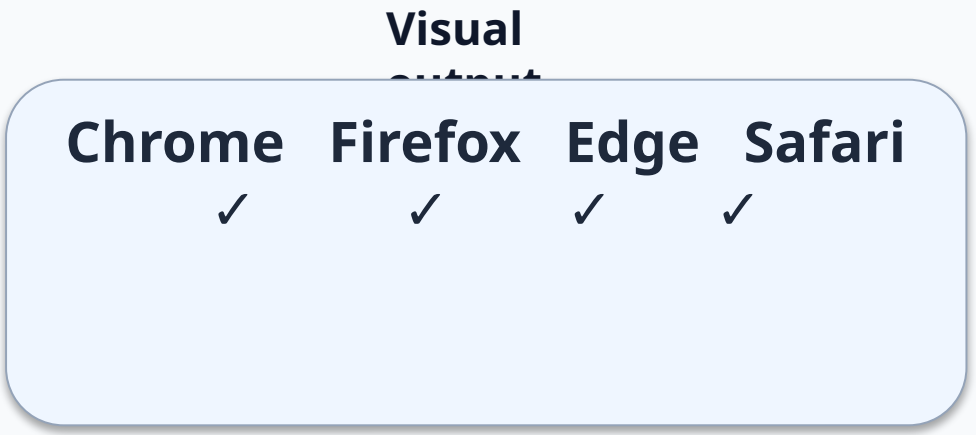
Modern CSS
transform: rotate(20deg);

Cross-Browser Compatibility

A website should work correctly across browsers such as Chrome, Firefox, Edge and Safari.

Browsers can differ in CSS feature support, rendering and default styles.

Developers should test important interfaces across target browsers.



How to Improve Compatibility?

Use standard CSS.

Check browser compatibility.

Provide fallbacks where appropriate.

Use feature queries when useful.

Test websites in multiple browsers.

Use responsive design and avoid unnecessary experimental features.

Visual output

Standard CSS + Fallback + Test

CSS3 Features: Quick Review

Visual output

Visual effects: border-radius, gradients, box-shadow, text-shadow, opacity

Transformations: translate, rotate, scale, skew, 3D transforms

Motion: transition, @keyframes, animation

Modern CSS: web fonts, icon fonts, CSS variables

Responsiveness and compatibility: @media, @supports, vendor prefixes, cross-browser testing

Effects | Motion | Fonts | Responsive | Compatibility

Key Concepts to Remember

Transform = what changes.

Transition = how smoothly it changes.

Animation = how it changes through multiple stages.

Media query = when different CSS should apply based on viewport/device conditions.

Feature query = whether the browser supports a CSS feature.

Visual output

Transform = change
Transition = smooth change
Animation = multiple stages